

RAG Pipeline Offloading via NVMe-oF for Edge LLM Inference



202255570 양윤성 youseong@pusan.ac.kr

202455325 나예은 nye6998@pusan.ac.kr

지도교수 안성용

목 차

1. 연구 배경 및 목적	1
1.1 연구 배경	1
1.2 연구 목적	1
2. 문제 정의 및 해결 방안	2
2.1 문제 정의	2
2.1.1 엣지 환경의 저장공간 제약	2
2.1.2 Vector DB 접근 시 I/O 병목	2
2.1.3 기존 접근 방식의 한계	2
2.2 해결 방안	3
2.2.1 분리형 Storage 구조	3
2.2.2 RAG 연산을 Storage Server로 이동	3
2.2.3 NVMe-oF 기반 RAG 및 Tokenization 오프로딩 구조	3
3. 시스템 구성	4
3.1 전체 시스템 아키텍처	4
3.2 Edge 아키텍처	4
3.2 Storage Server 아키텍처	5
4. 개발 환경 및 기술	6
5. 역할 분담	7

1. 연구 배경 및 목적

1.1 연구 배경

최근 대규모 언어 모델(LLM, Large Language Model)은 다양한 분야에서 활용되고 있다. 그러나 학습하지 않은 내용에 대한 환각(hallucinations) 문제를 가지며, 최신 정보를 반영하기 어렵다는 한계가 있다. 검색 증강 생성(RAG, Retrieval-Augmented Generation)은 이러한 한계를 보완하기 위해 외부 문서나 데이터베이스에서 관련 정보를 먼저 검색하고, 그 내용을 LLM의 응답 생성에 활용할 수 있도록 하는 기술이다.

그러나 RAG에서 사용하는 Vector DB의 규모가 커질수록 저장공간과 데이터 접근에 대한 부담이 증가한다. 특히 자원이 제한된 엣지 디바이스의 경우 대규모 Vector DB를 로컬에 유지하기 어려워 별도의 원격 Storage Server를 두는 구조가 필요하다. 이러한 경우 Edge와 Storage Server 사이의 네트워크를 통해 많은 데이터가 이동하게 되며 오버헤드가 발생할 수 있다. 본 연구는 이러한 엣지 환경의 자원 제약과 원격 Vector DB 접근 문제를 배경으로 기획되었다.

1.2 연구 목적

본 연구의 목적은 자원이 제한된 Edge Device에서 대규모 RAG 데이터베이스를 효율적으로 활용하기 위해, NVMe over Fabrics(NVMe-oF) 기반의 RAG 파이프라인 오프로딩 구조를 설계하고 구현하는 것이다. 기존 방식과 같이 Edge에서 LLM inference와 RAG를 모두 수행하는 대신, Storage Server에서 RAG와 tokenization을 수행하도록 역할을 분리하여 원격 데이터 접근 과정에서 발생하는 데이터 이동을 줄이고자 한다.

구체적인 목적은 다음과 같다

1. Edge와 Storage Server 간 RAG 파이프라인 오프로딩 구조 설계
2. NVMe-oF를 활용한 Edge-Storage Server 간 오프로딩 경로 구현
3. Storage Server에서 RAG 및 tokenization을 수행하고 결과를 Edge의 LLM inference와 연동하는 시스템 구현

2. 문제 정의 및 해결 방안

2.1 문제 정의

2.1.1 엣지 환경의 저장공간 제약

언어 데이터의 집합인 Corpus의 크기가 증가할수록 Vector DB의 규모도 같이 증가하고, 전체 Vector Index를 메모리에 유지하는 것이 어려워진다. 특히 저장공간이 제한된 Edge Device의 경우 대규모 Vector DB를 로컬에 저장하면서 LLM inference까지 동시에 수행하기 어렵다. 따라서 Vector DB를 Edge가 아닌 별도의 Storage Server에 두는 구조가 필요하다. 그러나 저장공간 문제를 해결하기 위해 데이터를 원격으로 분리하면 Edge와 Storage Server 사이에서 데이터 이동이 발생한다.

2.1.2 Vector DB 접근 시 I/O 병목

Vector DB의 크기가 커지면 Host Memory와 Storage 사이의 데이터 이동 자체가 중요한 병목이 될 수 있다. 특히 Vector DB가 Storage Server에 있고 Similarity Search가 Edge에서 수행되는 구조에서는 검색 과정에서 Storage를 접근이 발생한다. 이 경우 저장장치의 I/O와 네트워크 통신이 함께 발생하므로 추가적인 지연이 발생할 수 있다.

2.1.3 기존 접근 방식의 한계

Near-data processing(NDP)은 데이터가 저장된 위치와 최대한 가까운 곳에서 연산을 수행하는 방식이다. RAG는 Similarity Search 과정에서 Storage의 Vector DB에 반복적으로 접근하므로 NDP를 적용하기에 적합하다.

그러나 위의 방식은 별도의 하드웨어를 필요로 하기 때문에 일반적인 Edge환경에서 재현하거나 적용하기 어렵다. 또한 Retrieval 단계만 Storage로 이동하고, Tokenization은 여전히 Edge에서 수행하므로, 검색된 Augmented Text를 다시 Edge로 전송하면서 데이터 이동이 발생한다.

2.2 해결 방안

2.2.1 분리형 Storage 구조

대규모 Corpus와 Vector DB는 Edge 내부에 저장하지 않고 별도의 Storage Server에 배치한다. Edge는 LLM inference에 필요한 기능을 중심으로 구성하고, 대규모 데이터와 검색 처리는 Storage Server가 담당하도록 하여 Edge의 저장공간 부담을 줄인다.

2.2.2 RAG 연산을 Storage Server로 이동

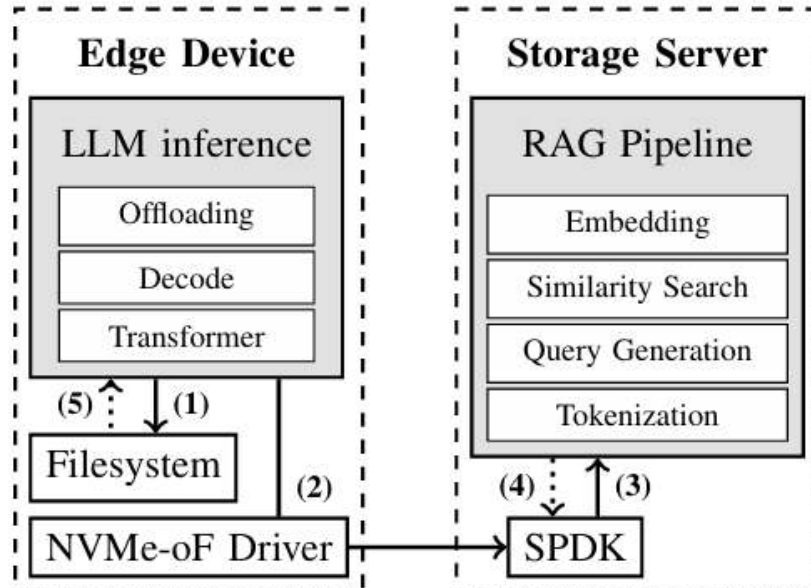
Embedding과 Similarity Search를 데이터가 위치한 Storage Server에서 수행한다. 이를 통해 Vector Index 탐색 과정에서 원격 Storage 접근이 네트워크를 반복적으로 통과하는 것을 줄여 네트워크 지연과 데이터 이동에 따른 오버헤드를 완화한다. 즉, Edge는 사용자의 질의를 Storage Server에 전달하고 처리된 결과를 전달받는 구조로 구성한다.

2.2.3 NVMe-oF 기반 RAG 및 Tokenization 오프로딩 구조

Edge와 Storage Server 간의 오프로딩 요청은 NVMe-oF를 기반으로 구성하며, Custom NVMe Command를 통해 질의와 결과를 전달한다. Storage Server에서는 SPDK 기반 NVMe-oF Target에 Custom Command Handler를 추가하여 RAG와 Tokenization을 수행하고, 생성된 Token을 Edge로 전달한다. 이를 통해 Retrieval만 Storage에서 수행하고 Tokenization은 Edge에서 처리하는 기존 방식과 달리, Tokenization까지 Storage Server에서 수행하여 Augmented Text를 다시 Edge로 전송하는 데이터 이동을 줄이고자 한다.

3. 시스템 구성

3.1 전체 시스템 아키텍처



전체 시스템은 크게 Edge Device와 Storage Server로 구성된다. Edge Device에서는 사용자 인터페이스와 LLM 추론을 담당하고, Storage Server는 Vector DB와 RAG pipeline을 담당한다. Edge와 Server는 NVMe-oF/TCP로 연결된다. 이 구조에서 Storage Server는 데이터와 가까운 위치에서 RAG 연산을 하는 NDP(근접 데이터 처리, Near-Data Processing) 노드로 동작한다.

3.2 Edge 아키텍처

- Edge Device가 사용자의 query를 받는다.
- offloading 결과 token을 저장하기 위한 placeholder file을 생성한다.
- 생성한 file의 LBA 범위와 query를 custom NVMe command에 포함하여 Server로 보낸다.
- Server가 보낸 custom command completion을 통해 길이를 확인한다.
- placeholder file을 Direct I/O로 읽어 page cache를 우회하고 결과 token을 가져온다.
- 읽어온 token을 LLM inference에 사용하여 Decode와 Transformer 작업을 수행한다.

3.3 Storage Server 아키텍처

- Edge Device로부터 query와 결과 token을 저장할 LBA 범위를 받는다.
- query를 이용하여 RAG pipeline을 수행한다.
 - 1) Embedding : 전달받은 query를 embedding vector로 변환한다.
 - 2) Similarity Search : 생성된 embedding을 이용해 Vector DB에서 관련 데이터를 검색한다.
 - 3) Query Generation : 검색 결과를 기존 query와 결합하여 LLM에 입력할 query를 생성한다.
 - 4) Tokenization : 생성된 query를 token sequence로 변환한다.
- RAG의 처리 결과로 생성된 token sequence를 Edge가 지정한 LBA 영역에 기록한다.
- custom command 완료 정보에 결과 길이를 포함하여 Edge에 전달한다.

4. 개발 환경 및 기술

구분	사용 기술	내용
Edge	Jetson Orin Nano	LLM inference를 수행하는 자원이 제한된 Edge
	Gemma 3 270M / llama.cpp	LLM 실행
통신	NVMe-oF/TCP	Server와 Edge 사이의 NVMe 통신
Server	PostgreSQL + pgvectorscale	Vector DB
	BGE-M3	embedding 생성
	SPDK	custom offload command handler 처리

5. 역할 분담

팀원	역할
양윤성	-llama.cpp 기반 Edge LLM inference 환경 구성 -RAG offloading 경로 구현 -Custom NVMe Command 구성 및 전송 -Storage Server에서 반환된 token sequence를 llama.cpp inference에 연결
나예은	-SPDK 기반 NVMe-oF Storage Server 환경 구성 -Custom NVMe Command handler 구현 -RAG pipeline 구현 -처리 결과 token을 Edge의 placeholder 영역에 기록